

Лабораторная работа №6

Разложение чисел на множители: р-метод Полларда

Кюнкриков Даниил Саналович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Идея ρ -метода Полларда	7
3.2	Практические замечания	8
4	Выполнение лабораторной работы	9
	Список литературы	12

Список иллюстраций

Список таблиц

1 Цель работы

Цель работы - Изучить принципы работы ρ -метода Полларда для факторизации и реализовать алгоритм на языке Python.

2 Задание

1. Реализовать ρ -метод Полларда для поиска нетривиального делителя составного числа n .
2. Продемонстрировать работу алгоритма на тестовом примере и получить разложение $n = p \cdot q$.

3 Теоретическое введение

Задача факторизации — разложение составного числа n на множители — лежит в основе оценки стойкости криптосистем, например RSA. Для чисел среднего размера применяются вероятностные алгоритмы, один из наиболее известных — ρ -метод Полларда [2; 3].

3.1 Идея ρ -метода Полларда

Выбирается отображение $f(x) = x^2 + c \pmod{n}$ и строится последовательность $x_{i+1} = f(x_i)$. Поскольку множество вычетов конечно, последовательность рано или поздно образует цикл (граф напоминает букву ρ). Для ускорения поиска коллизии применяется схема Флойда «черепаха-заяц»: - $a \leftarrow f(a)$,
- $b \leftarrow f(f(b))$.

На каждом шаге вычисляется:

$$d = \gcd(|a - b|, n).$$

Если $1 < d < n$, то найден нетривиальный делитель d ; если $d = n$, алгоритм перезапускают с другими параметрами [1; 3].

3.2 Практические замечания

Метод эффективен для поиска сравнительно небольших делителей и часто применяется как подалгоритм в более сложных схемах факторизации [2; 4].

4 Выполнение лабораторной работы

Программная реализация р-метода Полларда показана на Листинге №1.

4.0.1 Листинг 1 — р-метод Полларда (Python)

```
import math

def pollard_rho_factor(n: int, c: int = 5, x0: int = 1) -> int | None:
    #Возвращает нетривиальный делитель n или None, если попытка неудачна

    if n % 2 == 0:
        return 2

    def f(x: int) -> int:
        return (x * x + c) % n

    a = x0
    b = x0

    # Табличный вывод : номер шага, a, b, d
    print(" i\t a\t\t b\t\t d")
```

```

i = 0
while True:
    i += 1
    a = f(a)          # "черепаха"
    b = f(f(b))       # "заяц"
    d = math.gcd(abs(a - b), n)

    print(f"{i}\t {a}\t {b}\t {d}")

    if 1 < d < n:
        return d
    if d == n:
        return None

if __name__ == "__main__":
    n = int(input("n = ").strip())
    d = pollard_rho_factor(n)

    if d is None:
        print("\nПопытка найти делитель провалилась (d = n).")
    else:
        print(f"\nНетривиальный делитель: {d} и {n // d}")

```

Результат работы программы для $n = 1359331$ показан на ?@fig-1.

n = 135931			
i	a	b	d
1	6	41	1
1	41	123981	1
1	1686	87869	1
1	123981	68427	1
1	74955	38171	1
1	87869	120409	1
1	80366	112191	1
1	68427	41677	1
1	111039	75074	1
1	38171	24431	1
1	116788	11306	1
1	120409	13584	1
1	62757	19528	1
1	112191	74435	1
1	17679	83231	1
1	41677	105389	181
Нетривиальный делитель: 181 и 751			

Вы-

воды

1. Реализован р-метод Полларда для поиска нетривиального делителя составного числа.
2. Продемонстрирована работа алгоритма на тестовом примере: получено разложение $1359331 = 1181 \cdot 1151$.

Список литературы

1. *Brent R. P.* An Improved Monte Carlo Factorization Algorithm // BIT Numerical Mathematics. — 1980. — Т. 20. — С. 176—184. — DOI: 10.1007/BF01933190.
2. *Crandall R., Pomerance C.* Prime Numbers: A Computational Perspective. — 2-е изд. — Springer, 2005. — ISBN 0-387-25282-7.
3. *Pollard J. M.* A Monte Carlo Method for Factorization // BIT Numerical Mathematics. — 1975. — Т. 15, № 3. — С. 331—334. — DOI: 10.1007/BF01933667.
4. *Wagstaff S. S.* The Joy of Factoring. — American Mathematical Society, 2013. — ISBN 1-4704-1048-6.